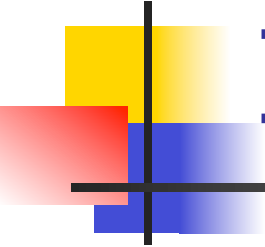


Corso di Crittografia

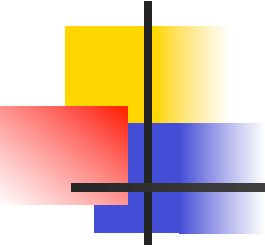
Prof. Dario Catalano

Firme Digitali



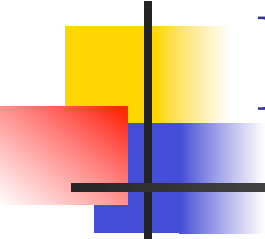
Introduzione

- Una firma digitale e' l'equivalente informatico di una firma convenzionale.
- Molto simile a MA, solo che qui abbiamo una struttura asimmetrica.
 - La chiave segreta e' utilizzata per creare firme
 - La chiave pubblica per verificarle



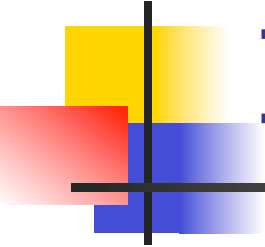
Differenza fondamentale

- Una firma digitale e' (in generale) non ripudiabile.
- La natura asimmetrica di tale primitiva fa si che solo un utente (colui che conosce la chiave segreta) possa firmare.
- Nei MAC non vi e' sostanziale differenza tra colui che autentica e colui che verifica.



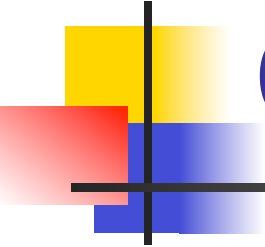
Formalmente

- Uno schema di firma digitale consiste nei seguenti algoritmi
 - Un algoritmo di generazione della chiave KeyGen
 - Un algoritmo di generazione della firma
 - Un algoritmo di verifica



In particolare

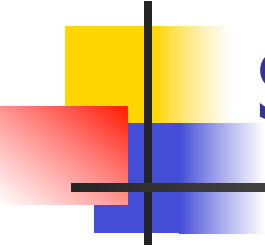
- L'algoritmo KeyGen (randomizzato) restituisce una coppia (vk, sk) appartenente all'insieme $Chiavi(k)$.
- L'algoritmo Sign (randomizzato, deterministico o a stati) prende in input sk e un messaggio m e restituisce una firma σ (o un simbolo speciale $\perp$)
- L'algoritmo Ver (deterministico) prende in input (vk, m, σ) e restituisce un bit.



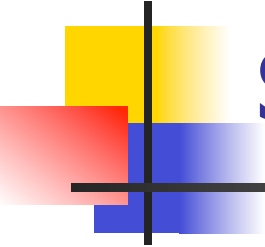
Considerazioni

- Nel caso in cui il sistema sia senza stati, bisogna specificare uno spazio di messaggi validi.
- Tale insieme non viene specificato nel caso di schemi di firma a stati, perché la validità di un messaggio può dipendere dallo stato.
- L'algoritmo Ver restituisce 1 per ogni firma correttamente generata.

Verso una definizione di sicurezza

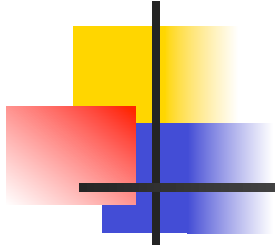


- Gran parte del lavoro lo abbiamo fatto per MA.
- Qui abbiamo una struttura piu' flessibile, dobbiamo solo adattare la definizione vista per MA al nuovo contesto.
- Ripassiamo qualcuna delle considerazioni fatte per MA

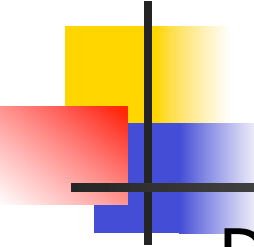


Verso una definizione di sicurezza (cont.)

- Obiettivo: rivelare ogni tentativo di modifica da parte di A.
- Il destinatario dovrebbe considerare autentico solo un msg M che non e' stato alterato.
- Se A produce (M, σ) tali che $\text{Ver}(\text{vk}, M, \sigma) = 1$, diciamo che A produce un *falso*.



- Anche nel nuovo contesto possiamo immaginare di dividere l'attacco in due fasi
 - Dapprima A richiede la firma di un certo numero di messaggi.
 - Quindi prova a costruire un "buon" falso
- Concediamo ad A solo un oracolo di firma.
 - A non ha alcun bisogno di un oracolo di verifica.



Definizione

■ DS: (KeyGen, Sign, VF)

$\text{Esp}_{\text{DS}}^{\text{uf-cma}} (A)$

$(vk, sk) \leftarrow_R \text{KeyGen}(); (M, \sigma) \leftarrow A^{\text{Sig}(sk, \cdot)}(vk);$

if $(\text{Ver}(vk, M, \sigma) == 1) \ \& \ (M \in \text{Messaggi}) \ \&$
 $(A \text{ non aveva richiesto } M \text{ all'oracolo})$

Return 1

else Return 0

$\text{Adv}^{\text{uf-cma}} (A) = \Pr[\text{Esp}_{\text{DS}}^{\text{uf-cma}} (A) = 1]$

DS garantisce non falsificabilità contro
attacchi a messaggio scelto e che
vantaggio è prossimo a zero $\forall A$
comp. limitato.

$$N = pq$$

e

$$\gcd(e, \phi(N)) = 1$$

d

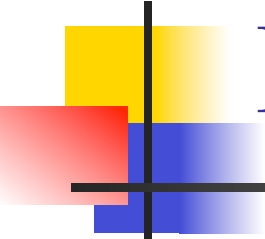
$$\phi(N) = (p-1)(q-1)$$

$$e \cdot d \equiv 1 \pmod{\phi(N)}$$

$$x \in \mathbb{Z}_N^*$$

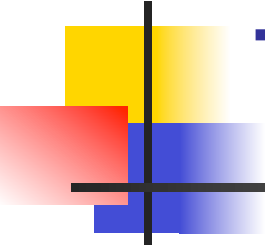
$$\text{RSA}[N, e](x) = y = x^e \pmod{N}$$

$$y^d \longrightarrow x$$



Firme RSA

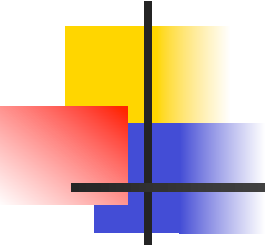
- La funzione RSA e' molto utilizzata anche come primitiva di base per costruire schemi di firme digitali.
- Partiremo da alcuni schemi di base e poi vedremo soluzioni via, via piu' sofisticate.
- In ogni caso, l'algoritmo di generazione delle chiavi per i sistemi basati su RSA e' praticamente sempre lo stesso.



Trapdoor Signatures

Idea: sfruttare la funzione trapdoor in modo inverso a quanto fatto per cifrare.

- Nel caso di RSA:
 - L'algoritmo di cifratura diventa l'algoritmo di verifica.
 - L'algoritmo di decifratura diventa l'algoritmo di firma
 - L'algoritmo di gen delle chiavi resta invariato.



Piu' precisamente

Sign _{$N, \underline{d}$} (M)

$$\mathcal{M} = \mathbb{Z}_N^*$$

If $M \notin \mathbb{Z}_N^*$ return $\perp$; $\underline{\sigma} \leftarrow M^d \bmod N$

Return σ

Ver _{N, e} (M, σ)

If $(M \notin \mathbb{Z}_N^* \vee \sigma \notin \mathbb{Z}_N^*)$ return $\perp$

if $\sigma^e = M \bmod N$ Return 1

else Return 0

$$M = \mathbb{Z}_N^* \setminus \{1\}$$

$$\text{Sign}_{N,d}(n)$$

if $(n \notin M)$ output $\perp$

$$\sigma \leftarrow n^d \bmod N$$

output σ

$$\text{Verify}_{N,e}(\sigma, n)$$

if $(n \notin M \vee \sigma \notin \mathbb{Z}_N^*)$ output $\perp$

if $\sigma^e \bmod N = n$ output $\perp$

else output 0

$$A(N, e)$$

$$\sigma \leftarrow \mathbb{Z}_N^* \setminus \{1\}$$

$$n \leftarrow \sigma^e \bmod N$$

$$(n, \sigma)$$

$$\mathcal{B}(N, e)$$

$$1. \quad H \in \mathcal{M}$$

$$2. \quad n_1, n_2 \in \mathcal{M}$$

$$n_1, n_2 \neq H$$

$$n_1 \cdot n_2 = n \pmod{N}$$

$$3. \quad O^{\text{Sign}}(n_1) \rightarrow G_1 \quad \left| \quad \underline{n_2 = n_1 \cdot n_1^{-1} \pmod{N}} \right.$$

$$4. \quad O^{\text{Sign}}(n_2) \rightarrow G_2$$

$$5. \quad \text{Output } (G = G_1 \cdot G_2 \pmod{N, n})$$

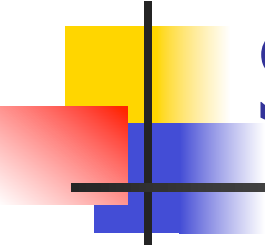
$$G_1^e \pmod{N} = n_1$$

$$G_2^e \pmod{N} = n_2$$

$$G_1^e \cdot G_2^e \pmod{N} = n_1 \cdot n_2 \pmod{N}$$

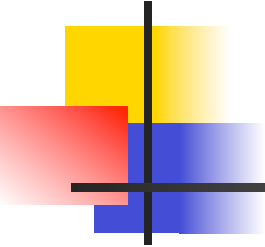
$$(G_1 \cdot G_2)^e \pmod{N} = n$$

$$G^e \pmod{N} = n$$



Sicurezza dello schema

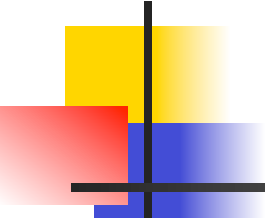
- Sappiamo che invertire RSA e' un problema difficile.
- E' questo sufficiente a garantire un qualche livello di sicurezza al nostro schema?



Hash e Inverti

- Dobbiamo trovare un modo per “distruggere” le proprietà algebriche di RSA.
- Inoltre sarebbe meglio poter assumere che M è una stringa qualunque e non un elemento di un gruppo.

Idea: modificare lo schema utilizzando una funzione hash



Hash e Inverti (cont)

Sign $_{N,d}(M)$

$y \leftarrow H_N(M); \quad // \quad H_N : \{0, 1\}^* \rightarrow \underline{Z_N^*}$

$\sigma \leftarrow y^d \bmod N$

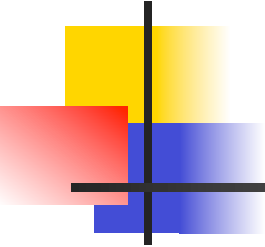
Return σ

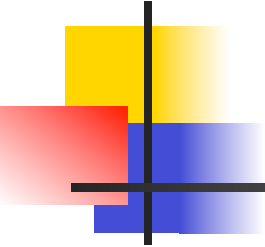
Ver $_{N,e}(M, \sigma)$

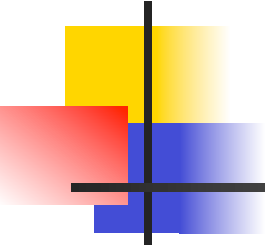
$y \leftarrow H_N(M); \quad y' = \sigma^e \bmod N;$

if $(y = y')$ Return 1

else Return 0

- 
-
- Questa soluzione risolve certamente il problema dell'arbitrarietà dei messaggi.
 - Intuitivamente sembra risolvere anche il secondo problema.
 - Una buona funzione hash, dovrebbe essere in grado di "distruggere" le proprietà algebriche che ci davano noia.
 - Però essa fa sorgere un problema che non avevamo in precedenza.

- 
-
- In prima istanza, potremmo isolare certe proprietà necessarie affinché lo schema possa funzionare.
 - Ad es. Resistenza alle collisioni
 - Il nostro obiettivo però rimane dimostrare che l'approccio è sicuro.
 - Limitarsi a condizioni necessarie può indurre all'errore.
 - Bisogna stabilire condizioni sufficienti.



Full Domain Hash

- Supponiamo di affrontare il problema in modo diverso
- Che succede se avessimo a disposizione una funzione hash perfetta (RO)?
 - Se hash-e-inverti ha problemi anche in questo caso sarebbe opportuno cambiare approccio.
 - Tale esperimento e' teorico perche' funz. Hash perfette non ne esistono.

Uf-cma di Hash e Inverti (nel ROM)

$\text{Esp}_{\text{DS}}^{\text{uf-cma}} (A)$

$((N,e),(p,q,d)) \leftarrow_R \text{KeyGen}^{\text{RSA}}();$

$H \leftarrow_R \text{Func}(\{0,1\}^*, Z_N^*);$

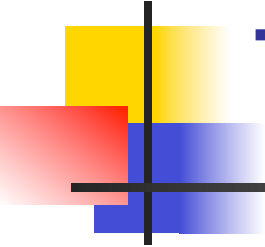
$(M,\sigma) \leftarrow A^{H(\cdot), \text{Sign}_N^H(\cdot)}(N,e);$

if $(\text{Ver}_{N,e}(M,\sigma) = 1) \ \& \ (M \in \text{Messaggi}) \ \&$
 $(A \text{ non aveva richiesto } M \text{ all'oracolo})$

Return 1

else Return 0

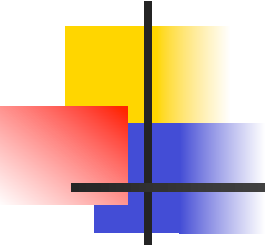
$$\text{Adv}^{\text{uf-cma}} (A) = \Pr[\text{Esp}_{\text{DS}}^{\text{uf-cma}} (A) = 1]$$



Teorema

- Sia $DS=(\text{KeyGen}_{\text{RSA}}, \text{Sign}, \text{Ver})$, lo schema descritto.
- Sia A un avversario che al più q_s richieste di firma e q_H richieste all'oracolo ($q_H > q_s$)
- Esiste un avversario I che usa risorse comparabili ad A , tale che

$$\text{Adv}^{\text{uf-cma}}(A) \leq q_H \text{Adv}^{\text{ow-rsa}}(I)$$



EC - DSA

Key Gen

$$q \quad G \quad g$$
$$x \leftarrow \{1, \dots, q\}$$

$$h \leftarrow g^x$$

$$H: \{0, 1\}^* \rightarrow \{1, \dots, q\}$$

$$vk \equiv (G, g, q, h, H)$$

$$\mathcal{M} = \{0, 1\}^*$$

$$sk = x$$

$\text{Sig}_{\text{sk}}(m)$

$$k \leftarrow^R \{1, \dots, q\}$$

$$R \leftarrow g^k ; \quad r \leftarrow R \bmod q$$

$$s \leftarrow k^{-1} (H(m) + r x) \bmod q$$

output (r, s)

$\text{Ver}(vk, m, (r, s))$

$$w \leftarrow s^{-1} \bmod q$$

$$u_1 \leftarrow g^{w H(m)}$$

$$u_2 \leftarrow h^{w r}$$

$$v \leftarrow u_1 \cdot u_2 ;$$

if $(v \bmod q == r)$ return 1
else return 0

$$\begin{aligned} v &= u_1 \cdot u_2 = g^{w H(m)} h^{w r} = g^{w H(m)} \cdot g^{w x r} \\ &= g^{w (H(m) + x r)} \end{aligned}$$

$$v = g^w (H(m) + xr)$$

$$w = s^{-1} \text{ mod } q = [k^{-1} (H(m) + xr)]^{-1} \\ = k (H(m) + xr)^{-1}$$

$$v = g^{k (H(m) + xr)^{-1} (H(m) + xr)} \\ = g^k$$

$$v \text{ mod } q = g^k \text{ mod } q$$

Schorr's Signature

KeyGen is same as prena (EC-DSA)

$\text{Sign}_{sk}(m)$

$$r \leftarrow \{1, \dots, q\}$$

$$R \leftarrow g^r$$

$$c \leftarrow H(R, vk, m)$$

$$z \leftarrow r + c \times m \bmod q$$

output (R, z)

$$\text{Ver}_{vk}((R, z), m)$$

$$c = H(R, vk, m)$$

if $(R \cdot h^c = g^z)$ output 1
 else output 0

$$\begin{aligned} R \cdot h^c &= g^r \cdot h^c = g^r \cdot (g^x)^c = \\ &= g^{r + cx} = g^z \end{aligned}$$